

Family Recipe Meal Planner - Project Plan

Project Overview

Project Name: Family Recipe Vault & Weekly Meal Planner

Duration: 24-28 weeks (6-7 months)

Time Commitment: 5-10 hours/week

Total Effort: ~160-200 hours

Budget: \$0 (100% free tier services)

Project Objectives

Primary Goals

1. Build a functional recipe management system for 1-20 family users
2. Implement weekly meal planning with calendar interface
3. Generate smart shopping lists with ingredient aggregation
4. Learn enterprise data engineering tools (PySpark, Kafka, AWS, Airflow)
5. Create portfolio-worthy project demonstrating scalable architecture

Success Metrics

- ☒ Family actively using the meal planner weekly
 - ☒ 100+ recipes in database
 - ☒ Working data pipeline with PySpark processing
 - ☒ Portfolio project ready for job interviews
 - ☒ Deployable demo at live URL
-

Technical Stack

Production Application (Free Tier)

- **Frontend:** React.js with Tailwind CSS
- **Hosting:** Vercel (free)
- **Backend:** Supabase (PostgreSQL + REST API)
- **Authentication:** Supabase Auth
- **Image Storage:** Cloudinary (free tier)
- **Domain:** Optional .online domain or free Vercel URL

Data Engineering Learning Stack (Free Tier)

- **PySpark:** Databricks Community Edition
- **Kafka:** Upstash Kafka (free tier) or Local Docker
- **Workflow Orchestration:** Apache Airflow (local Docker)
- **Cloud:** AWS Free Tier (S3, Glue, Athena)
- **Version Control:** GitHub

- **CI/CD:** GitHub Actions
-

Project Phases & Timeline

PHASE 1: Foundation & Setup (Weeks 1-3)

Duration: 3 weeks | **Hours:** 15-20 hours

Week 1: Environment Setup

- ☐ Set up GitHub repository
- ☐ Create Vercel account and connect GitHub
- ☐ Set up Supabase project
- ☐ Configure Cloudinary account
- ☐ Install Node.js, npm, React development tools
- ☐ Set up local development environment

Deliverable: Development environment ready

Week 2: Learn React Basics

- ☐ Complete React tutorial (official docs)
- ☐ Understand components, props, state, hooks
- ☐ Practice with useState, useEffect
- ☐ Learn React Router basics
- ☐ Understand form handling in React

Deliverable: Comfortable building React components

Week 3: Database Design

- ☐ Design database schema (recipes, users, meal_plans, ingredients)
- ☐ Set up Supabase tables
- ☐ Configure Row Level Security (RLS) policies
- ☐ Test CRUD operations via Supabase interface
- ☐ Create initial seed data (10 sample recipes)

Deliverable: Database schema implemented in Supabase

PHASE 2: MVP - Recipe Management (Weeks 4-8)

Duration: 5 weeks | **Hours:** 25-35 hours

Week 4: Basic UI Framework

- ☐ Create React app with Vite
- ☐ Set up Tailwind CSS
- ☐ Build header/navigation component
- ☐ Create home page layout
- ☐ Set up routing (home, recipes, add recipe, recipe detail)

Deliverable: Basic app structure with navigation

Week 5: Recipe CRUD - Create & Read

- ☐ Build "Add Recipe" form (name, cuisine, cook time, ingredients, instructions)
- ☐ Connect to Supabase API
- ☐ Create recipe list view with cards
- ☐ Implement recipe search/filter
- ☐ Add image upload to Cloudinary

Deliverable: Can add and view recipes

Week 6: Recipe CRUD - Update & Delete

- ☐ Build recipe detail page
- ☐ Implement edit functionality
- ☐ Add delete with confirmation
- ☐ Handle image updates
- ☐ Error handling and validation

Deliverable: Full CRUD operations working

Week 7: User Authentication

- ☐ Implement Supabase Auth (email/password)
- ☐ Create login/signup pages
- ☐ Add protected routes
- ☐ Associate recipes with users
- ☐ Build user profile page

Deliverable: Multi-user support with auth

Week 8: Testing & Deployment

- ☐ Test all CRUD operations
- ☐ Fix bugs and edge cases
- ☐ Deploy to Vercel
- ☐ Configure environment variables
- ☐ Share with family for feedback

Deliverable: 🚀 Live MVP at [yourapp.vercel.app](#)

PHASE 3: Meal Planner Feature (Weeks 9-14)

Duration: 6 weeks | **Hours:** 30-40 hours

Week 9: Meal Plan Database & API

- ☐ Design meal plan schema (meal_plans, meal_plan_days tables)
- ☐ Create Supabase tables
- ☐ Build API functions for meal plan CRUD
- ☐ Test creating/retrieving weekly meal plans

Deliverable: Backend ready for meal planning

Week 10: Calendar UI

- ☐ Build week calendar grid component
- ☐ Display current week

- ☐ Add prev/next week navigation
- ☐ Show selected recipes in calendar slots
- ☐ Style calendar with Tailwind

Deliverable: Calendar display working

Week 11: Recipe Selection Interface

- ☐ Build recipe search/filter modal
- ☐ Implement autocomplete search
- ☐ Add filter by cuisine, cook time, difficulty
- ☐ Create recipe picker for each day

Deliverable: Can search and select recipes

Week 12: Drag & Drop (Optional - Can Skip)

- ☐ Install react-beautiful-dnd or dnd-kit
- ☐ Implement drag recipe to calendar day
- ☐ Handle drop events
- ☐ Update database on drop

Alternative: Use dropdown/button selection instead

Deliverable: Interactive meal planning

Week 13: Shopping List Generation

- ☐ Extract ingredients from selected recipes
- ☐ Build shopping list view
- ☐ Display ingredients grouped by recipe
- ☐ Add print/export functionality

Deliverable: Basic shopping list works

Week 14: Testing & Refinement

- ☐ Test meal planning workflow end-to-end
- ☐ Fix bugs and UX issues
- ☐ Add loading states and error handling
- ☐ Mobile responsive design
- ☐ Deploy updated version

Deliverable: 🎉 Functional meal planner live

PHASE 4: Big Data Tools - Introduction (Weeks 15-18)

Duration: 4 weeks | **Hours:** 20-30 hours

Week 15: PySpark Fundamentals

- ☐ Sign up for Databricks Community Edition
- ☐ Complete PySpark tutorial
- ☐ Learn RDDs, DataFrames, transformations
- ☐ Practice filtering, grouping, aggregations

- ☐ Understand lazy evaluation

Deliverable: Comfortable with PySpark basics

Week 16: AWS Setup & S3

- ☐ Create AWS Free Tier account
- ☐ Set up billing alerts (\$5, \$10, \$20)
- ☐ Learn S3 basics (buckets, objects)
- ☐ Upload sample recipe data to S3
- ☐ Practice querying with AWS Athena

Deliverable: Data stored in S3, queryable via Athena

Week 17: Kafka Basics

- ☐ Sign up for Upstash Kafka free tier (OR install local Docker Kafka)
- ☐ Learn Kafka concepts (topics, producers, consumers)
- ☐ Create a topic for recipe events
- ☐ Write Python producer to send recipe data
- ☐ Write Python consumer to receive data

Deliverable: Working Kafka producer/consumer

Week 18: Airflow Introduction

- ☐ Install Airflow locally with Docker
- ☐ Learn DAG concepts
- ☐ Create simple DAG to run daily
- ☐ Schedule a Python script
- ☐ Monitor DAG execution

Deliverable: Basic Airflow DAG running

PHASE 5: Smart Features with Data Engineering (Weeks 19-24)

Duration: 6 weeks | **Hours:** 35-45 hours

Week 19: Ingredient Normalization Pipeline

- ☐ Export recipes from Supabase to S3 as JSON
- ☐ Write PySpark job in Databricks
- ☐ Parse ingredient strings (quantity, unit, name)
- ☐ Normalize units (tsp → cups, etc.)
- ☐ Handle variations ("all-purpose flour" vs "flour")

Deliverable: PySpark job normalizes ingredients

Week 20: Smart Shopping List Aggregation

- ☐ PySpark job to aggregate ingredients across recipes
- ☐ Group by ingredient name
- ☐ Sum quantities with same units
- ☐ Convert units when possible (12 tsp → 4 tbsp)
- ☐ Output consolidated shopping list

Deliverable: Aggregated shopping list from PySpark

Week 21: Event-Driven Architecture

- ☐ Send "meal plan created" event to Kafka
- ☐ Kafka consumer triggers PySpark job
- ☐ Process ingredients in near-real-time
- ☐ Store results back to Supabase
- ☐ Update shopping list automatically

Deliverable: Event-driven ingredient processing

Week 22: AWS Glue Integration

- ☐ Create Glue crawler for S3 recipe data
- ☐ Set up Glue Data Catalog
- ☐ Write Glue ETL job (Python/PySpark)
- ☐ Transform and enrich recipe data
- ☐ Query with Athena

Deliverable: Glue ETL pipeline working

Week 23: Recipe Recommendations

- ☐ PySpark job to calculate recipe similarity
- ☐ Use ingredient overlap as similarity metric
- ☐ Suggest diverse recipes for weekly plan
- ☐ Display recommendations in UI
- ☐ "People also planned..." feature

Deliverable: Basic recommendation engine

Week 24: Airflow Orchestration

- ☐ Create DAG for nightly batch processing
- ☐ Schedule: Extract recipes → Process with PySpark → Update recommendations
- ☐ Add error handling and retries
- ☐ Set up email alerts on failure
- ☐ Monitor execution

Deliverable: Automated data pipeline

PHASE 6: Polish & Portfolio (Weeks 25-28)

Duration: 4 weeks | **Hours:** 20-25 hours

Week 25: Advanced Features (Pick 2-3)

- ☐ Pantry tracking (what you already have)
- ☐ Nutrition info extraction
- ☐ Recipe ratings and comments
- ☐ Recipe version history
- ☐ Weekly meal plan templates
- ☐ Budget tracking per meal plan

Deliverable: 2-3 polish features added

Week 26: Optimization & Performance

- ☐ Optimize PySpark jobs (caching, partitioning)
- ☐ Add Redis caching for frequent queries
- ☐ Optimize database queries
- ☐ Compress images
- ☐ Performance testing

Deliverable: Fast, optimized app

Week 27: Documentation

- ☐ Write comprehensive README
- ☐ Architecture diagrams (draw.io or Lucidchart)
- ☐ Document data pipeline flow
- ☐ API documentation
- ☐ Set up instructions for local development
- ☐ Create demo video (Loom)

Deliverable: Portfolio-ready documentation

Week 28: Final Polish & Launch

- ☐ Mobile responsive design check
- ☐ Cross-browser testing
- ☐ Security audit (env vars, API keys)
- ☐ Final bug fixes
- ☐ Announce to family
- ☐ Share on LinkedIn/GitHub

Deliverable: 🚀 Production-ready application

Project Milestones

Milestone	Week	Description
🔗 M1: Dev Environment Ready	Week 3	All tools installed, database schema designed
🍷 M2: MVP Launch	Week 8	Recipe CRUD live, family can add recipes
📅 M3: Meal Planner Live	Week 14	Calendar feature working, can plan weekly meals
🧰 M4: Data Tools Learned	Week 18	Comfortable with PySpark, Kafka, Airflow, AWS
🤖 M5: Smart Features	Week 24	Automated pipelines, ingredient aggregation working
🎓 M6: Portfolio Ready	Week 28	Documented, polished, demo-able project

Risk Management

Potential Risks & Mitigations

Risk	Probability	Impact	Mitigation
AWS costs exceed free tier	Medium	Low	Set billing alerts; shut down resources when not using

Risk	Probability	Impact	Mitigation
Complexity overwhelms progress	High	High	Build MVP first; add complexity gradually
Time commitment insufficient	Medium	High	Adjust timeline; skip optional features
Free tier limitations hit	Low	Medium	Have backup plans (DuckDB instead of Redshift)
Learning curve steeper than expected	Medium	Medium	Extend timeline by 4-8 weeks if needed
Family not interested in using it	Low	Low	Still valuable as portfolio project

Weekly Time Budget

Recommended Schedule (7.5 hours/week average):

- **Weeknights:** 1 hour/night $\times$ 3 nights = 3 hours
- **Weekend:** 2-hour session + 2.5-hour session = 4.5 hours

Flexibility: 5-10 hours/week depending on complexity

Success Criteria

Must-Have (MVP)

- ☒ User can CRUD recipes
- ☒ User can create weekly meal plans
- ☒ Shopping list generates automatically
- ☒ App is deployed and accessible online
- ☒ Multi-user support with authentication

Should-Have (Enhanced)

- ☒ PySpark processing for ingredients
- ☒ Kafka event streaming
- ☒ AWS data lake (S3 + Athena)
- ☒ Airflow scheduling
- ☒ Recipe recommendations

Nice-to-Have (Polish)

- 🍷 Drag & drop calendar
- 🍷 Pantry tracking
- 🍷 Nutrition analytics
- 🍷 Mobile app (React Native)
- 🍷 Recipe import from URLs

Tools & Resources

Learning Resources

- **React:** react.dev
- **Supabase:** supabase.com/docs
- **PySpark:** [Databricks Academy](https://databricksacademy.org/)
- **Kafka:** [Kafka Documentation](https://kafka.apache.org/documentation/)
- **AWS:** [AWS Free Tier Guide](https://aws.amazon.com/free/)
- **Airflow:** [Airflow Tutorial](https://airflow.apache.org/tutorial.html)

Design Tools

- **Wireframes:** Figma (free)
- **Architecture Diagrams:** Draw.io (free)
- **Icons:** Lucide React (free)

Monitoring

- **Uptime:** UptimeRobot (free)
- **Analytics:** Vercel Analytics (free)
- **Error Tracking:** Sentry (free tier)

Portfolio Presentation

GitHub README Template

markdown

Family Recipe Vault & Meal Planner

A full-stack recipe management and meal planning application with enterprise-grade data pipelines.

🌀 Features

- Recipe CRUD with image uploads
- Weekly meal planning with calendar interface
- Smart shopping list with ingredient aggregation
- Real-time updates via Kafka event streaming
- Automated data processing with PySpark
- Recipe recommendations engine

🛠️ Tech Stack

****Frontend:**** React, Tailwind CSS, Vercel

****Backend:**** Supabase (PostgreSQL), REST API

****Data Engineering:**** PySpark, Kafka, AWS (S3, Glue, Athena), Airflow

****Storage:**** Cloudinary (images), S3 (data lake)

🏗️ Architecture

[Insert architecture diagram]

🚀 Live Demo

[your-app.vercel.app]

📺 Video Walkthrough

[Loom video link]

Interview Talking Points

1. **Scale:** "Designed for 20 users but architected to scale to thousands"
2. **Trade-offs:** "Used PySpark for learning, but would use simpler stack for production at this scale"
3. **Real users:** "Family actively uses it weekly - incorporated their feedback"
4. **Data pipeline:** "Implemented event-driven architecture with Kafka and automated workflows with Airflow"
5. **Cloud-native:** "Leveraged AWS services (S3, Glue, Athena) following best practices"

Next Steps

Immediate Actions (This Week)

1. ☐ Star this project plan in your notes
2. ☐ Set up GitHub repository
3. ☐ Create Vercel account
4. ☐ Create Supabase account
5. ☐ Block out weekly time slots in calendar

Week 1 Kickoff Checklist

- ☐ Install Node.js and npm
- ☐ Set up local development environment
- ☐ Initialize React app

-  Connect GitHub to Vercel
 -  Create first commit: "Initial setup"
-

Project Tracking

Recommended Tools:

- **Task Management:** Notion (free) or Trello (free)
- **Time Tracking:** Toggl (free tier)
- **Notes:** Obsidian (free) or Notion

Weekly Review Questions:

1. Did I complete this week's deliverable?
 2. What blocked my progress?
 3. What do I need to learn next week?
 4. Am I still on timeline or need to adjust?
-

Contact & Support

When Stuck:

- Check official documentation first
- Search Stack Overflow
- Ask in Discord communities (Reactiflux, Data Engineering)
- Reddit: r/reactjs, r/dataengineering

Code Review:

- Post GitHub repo for feedback on r/webdev
 - Share progress on LinkedIn for accountability
-

Project Status:  Ready to Start

Next Milestone: M1 - Dev Environment Ready (Week 3)

Let's build something awesome! 